

Week 1

Intelligent Agents

What is Intelligence?

Remember the past. Learn from it.

Understand the present. Be aware of the world around you.

Imagine the future. Work towards your goal.

Characteristics

Persistent

Autonomous

Proactive

Goal-directed

Information Processing View of AI

An Autonomous agent

A decade of Machine Learning

Neural Networks

The Hidden Layer

Human Cognitive Architecture

Artificial Intelligence and Machine Learning

Knowledge and Reasoning

Representation

AI = Automation?

Problem-Solving

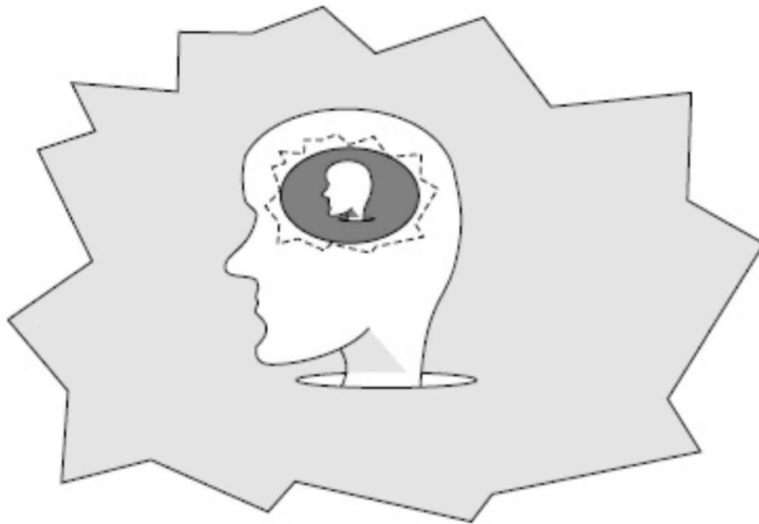
Dealing with Simple Problems

Humankind - A problem-solving species

Architecture of Problem-Solving

Intelligent Agents

An intelligent agent in a world carries a model of the world in its 'head'. The model maybe an abstraction. A self-aware agent would model itself in the world model.



Deeper awareness may require that the agent represents (be aware of itself modelling the world.

What is Intelligence?

Remember the past. Learn from it.

- Memory and Experience - Case-based reasoning
- Learn a model - Machine Learning
- Recognise faces, objects and patterns - Deep Neural Networks

Understand the present. Be aware of the world around you.

- Create a model of the world - knowledge representation
- Make inferences from what you know - logic and reasoning

Imagine the future. Work towards your goal.

- Trial and Error - heuristic search
- Goals, plans and actions - automated planning

Characteristics

Persistent

it is there all the time

Autonomous

acts autonomously

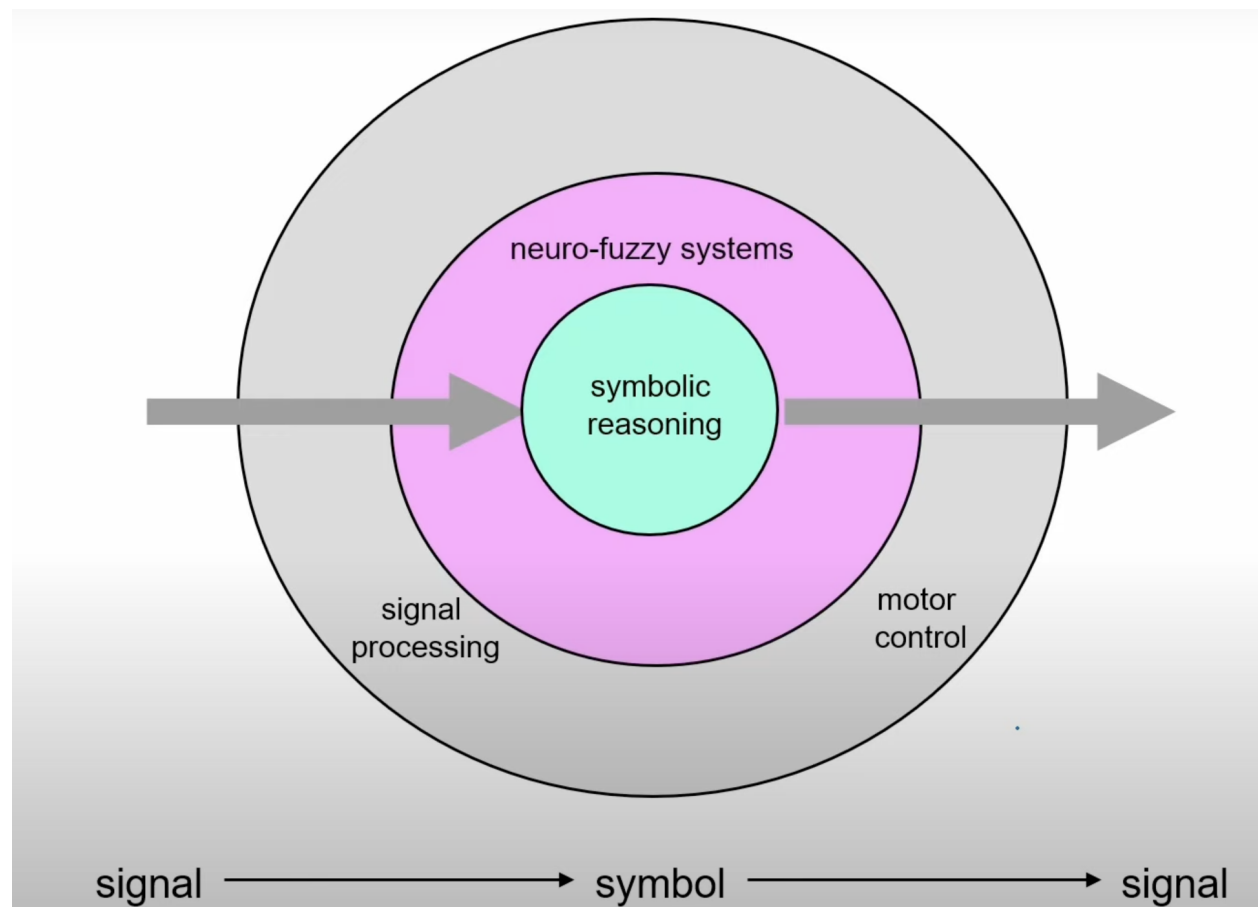
Proactive

can decide what goals to achieve next

Goal-directed

once given a goal, achieves it

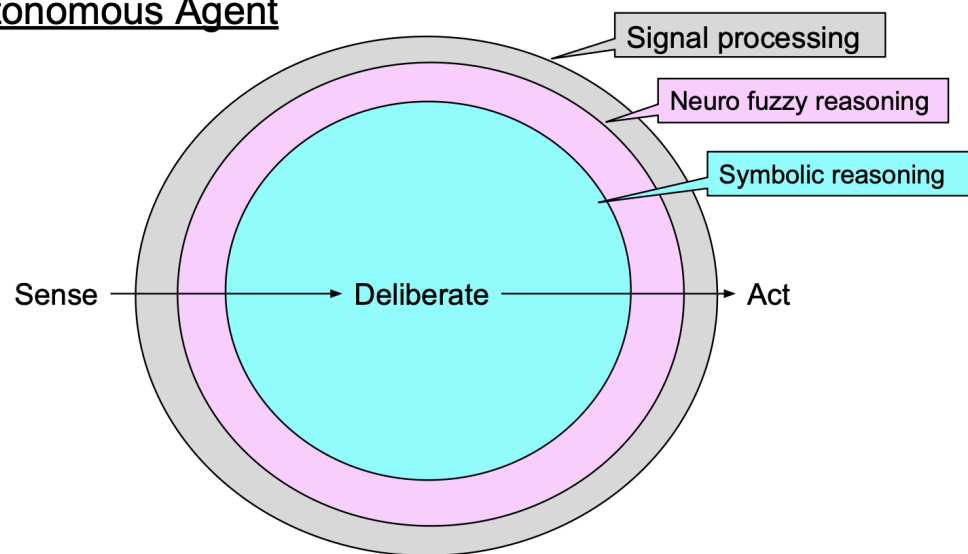
Information Processing View of AI



A model of a cognitive agent. The innermost circle represents Classical AI which is concerned with symbolic reasoning. This is encapsulated in neuro-fuzzy systems that produce and consume the symbols. The outermost layer deals with the external world directly, processing signals and producing motor activity.

An Autonomous agent

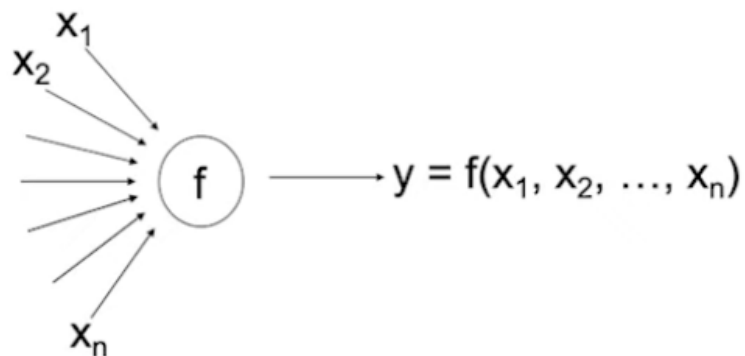
An Autonomous Agent



A decade of Machine Learning

Neural Networks

A neuron is simply a device that computes a simple function of the inputs it receives.

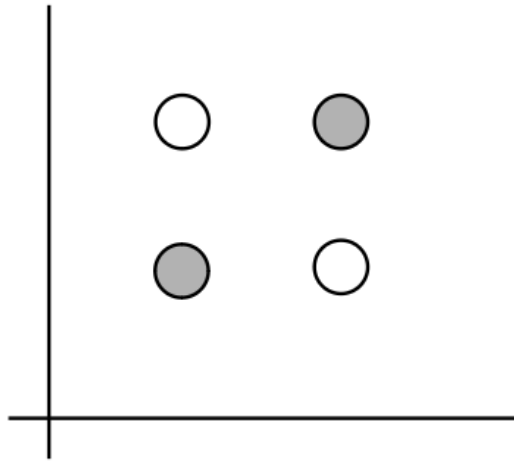


A collection of interconnected neurons can do complex computations. Sometimes, referred to as emergent systems.

- The Perceptron, invented by Frank Rosenblatt, was a single-layer neural network designed to be a binary classifier. McCulloch and Pitts proposed the *computational model*

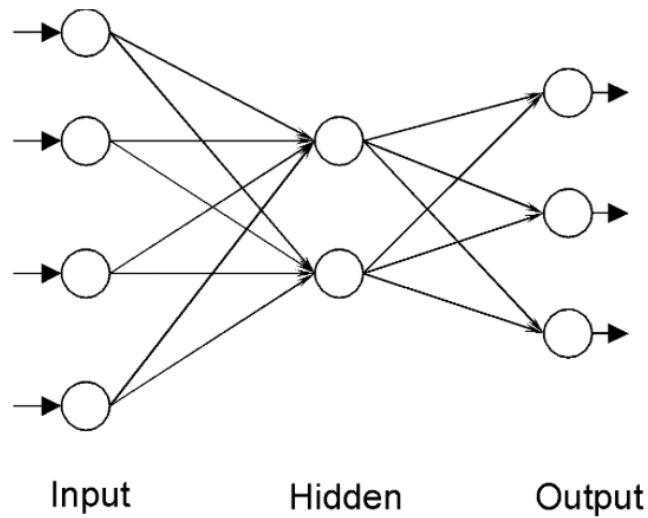
of the artificial neuron.

- In 1958 Minsky and Papert showed its limitations – it could only classify linearly separable classes.
- For example, one cannot draw a line to separate the shaded circles, data from class A, from the empty ones, from class B

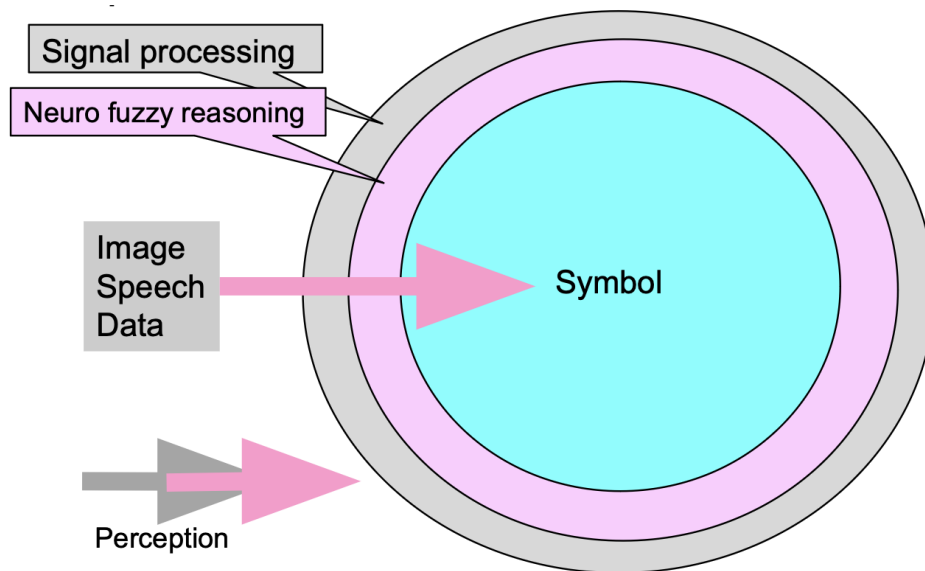


The Hidden Layer

- In 1986 Rumelhart, Hinton & Williams showed that a multi-layer perceptron could be trained to learn any non-linear classifier
- They also popularized the *Backpropagation Algorithm* which fed the error at the output layer back via the hidden layer, adjusting the weights of the connections.
- Hinton persevered with neural networks and showed in 2012 that (deep) networks with many hidden layers achieved phenomenal success in computer vision - recognizing thousands of types of objects

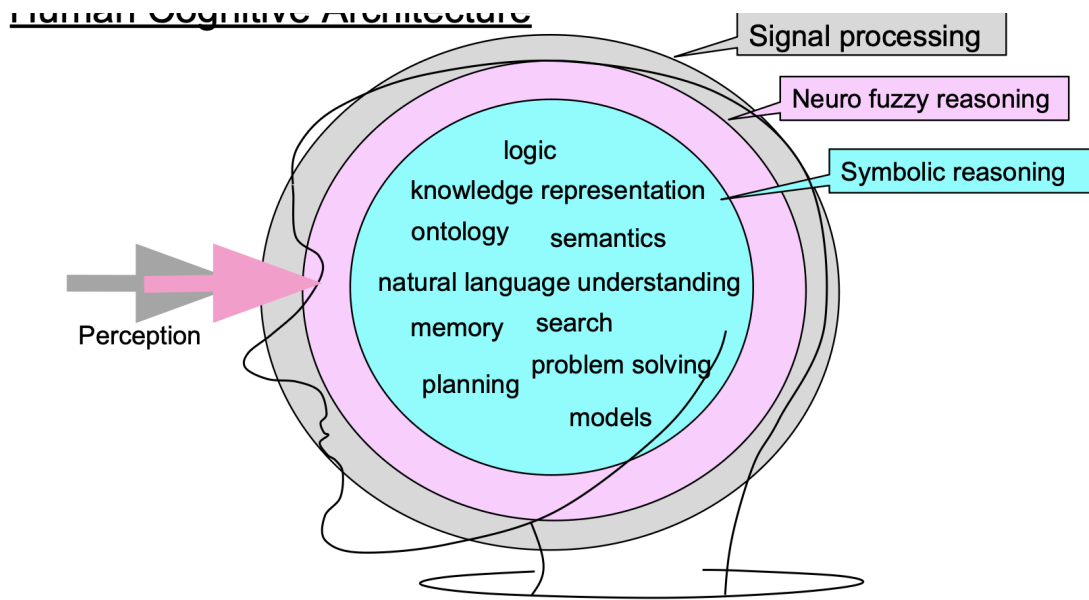


A feedforward (information flows from left to right) Artificial Neural Network



A symbol is a perceptible something that stands for something else. It has no intrinsic meaning.

Human Cognitive Architecture



Artificial Intelligence and Machine Learning

AI → symbolic knowledge representation and problem-solving

ML → making sense of data

- Data → Information (big data, recommender systems, predictive analytics....)
- Data → Classification (deep learning, images and language....)

Knowledge and Reasoning

What does the agent know*

and

what else does the agent know as a consequence of what it knows?

*We are concerned with explicit declarative knowledge and not implicit procedural or tacit knowledge

Reasoning- The manipulation of symbols in a meaningful manner.

Representation

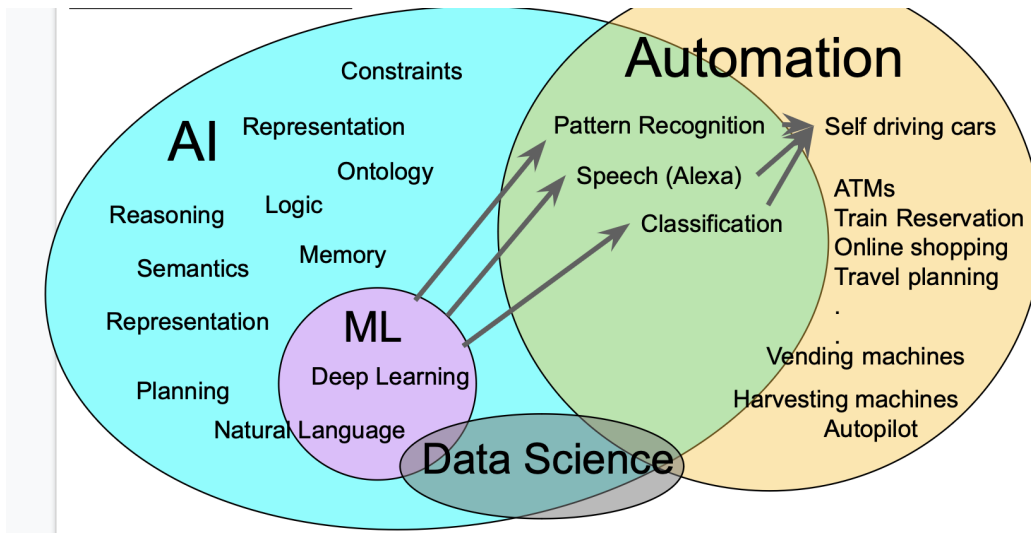
Semiotics: A **symbol** is something that **stands for** something else

- All languages, both spoken and written, are semiotic systems

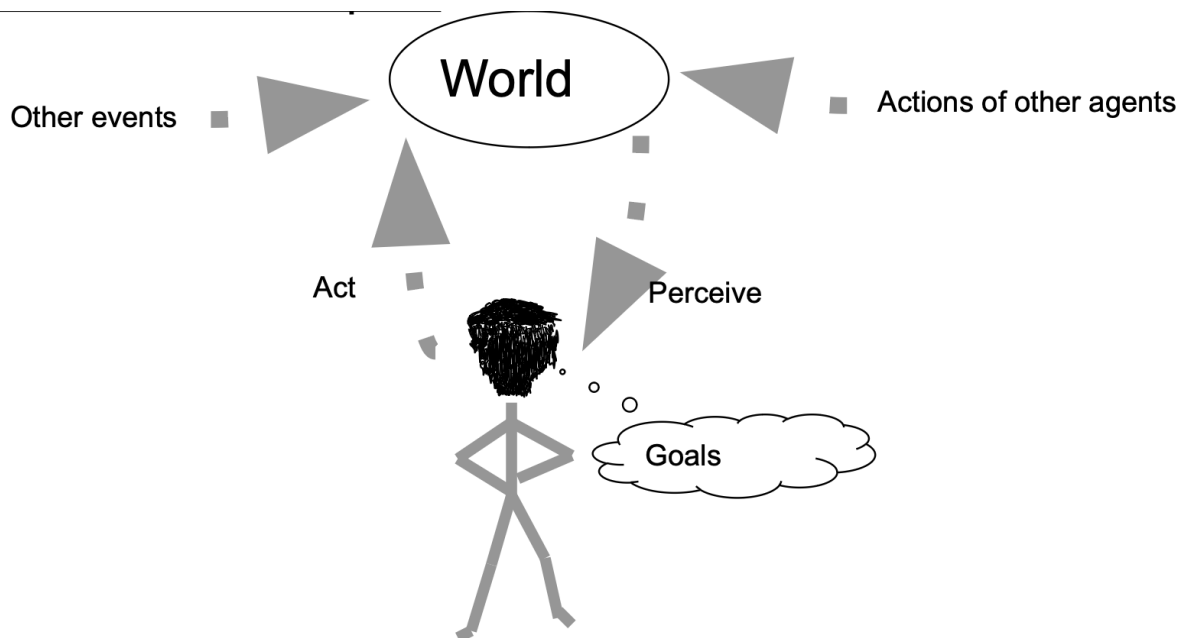
Biosemiotics: How complex behaviour emerges when simple systems interact with each other through signs.

- For example, pheromone trails left by ants

AI = Automation?



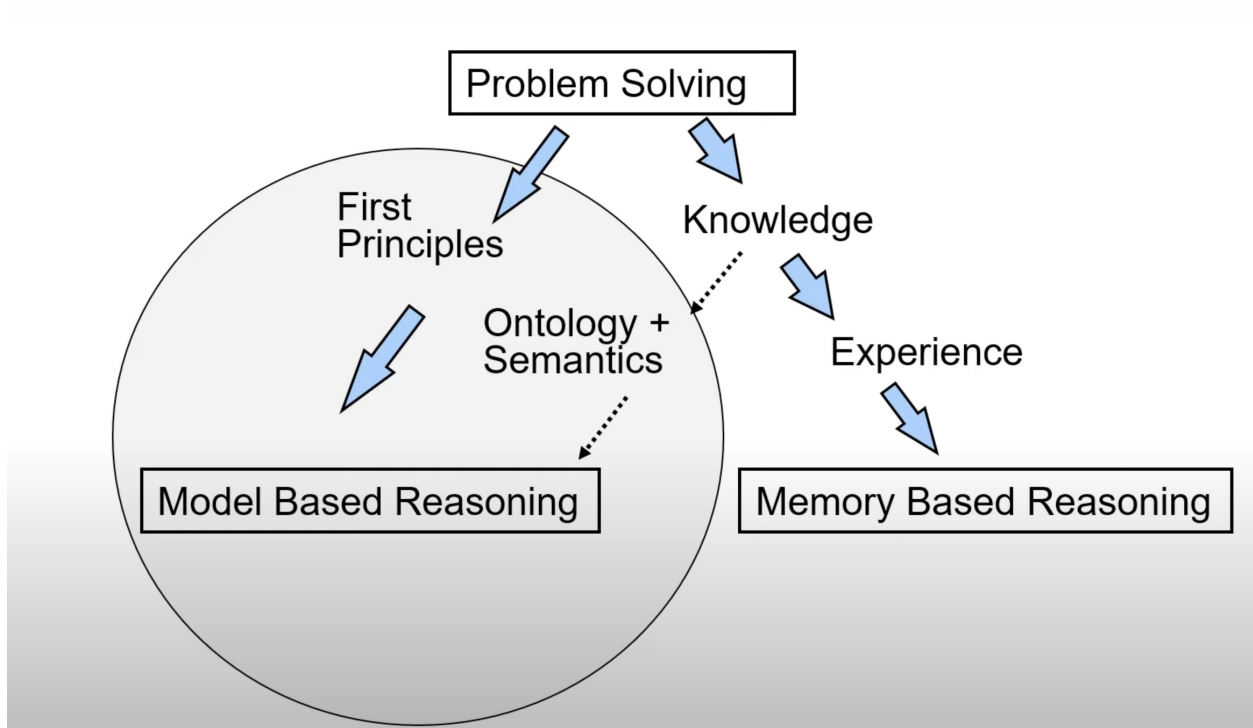
Problem-Solving



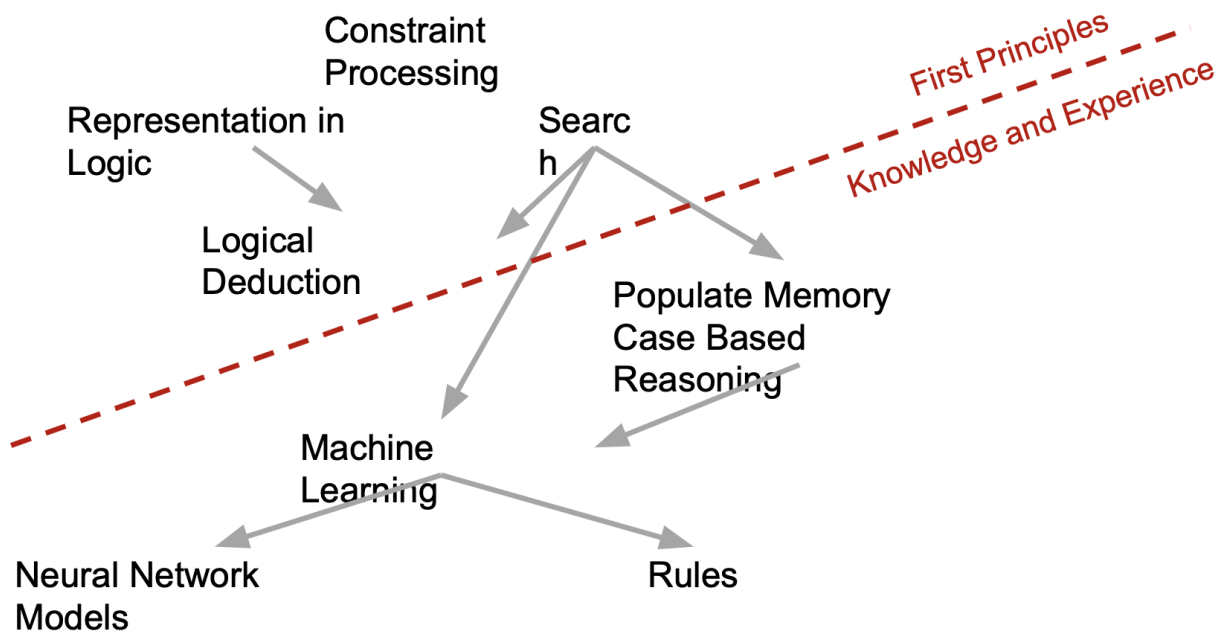
Dealing with Simple Problems

- The world is static.
- The world is completely known.
- Only one agent changes the world.
- Actions never fail.
- Representation of the world is taken care of

Humankind - A problem-solving species



Architecture of Problem-Solving



Week 2

Search Methods

Brute Force

Informed Search

General Search - Types

State Space Search / Solution Space Search

State Space Search → Graph Search

Constraint Processing

General Search → Algorithms

Functions

Example

Simple Search Algorithm 1

Simple Search Algorithm 2

Planning Problems

Node Pairs

Deterministic Search Algorithms

LIST Notations

TUPLE operators

LIST and TUPLES

Depth First and Breadth First Search

Ancillary Functions

Remove Duplicates

Occurs in

Make Pairs

Reconstruct Path

Depth First Search

Breadth First Search

Analysis of BFS and DFS

Behaviour

Running Time

Time Complexity

Space

Search Frontiers

Comparison - Summary

Best of Both

Depth Bounded DFS (DBDFS)

Algorithm
DBDFS -2
Depth First Iterative Deepening (DFID)
Closed nodes
Without closed nodes
Cost of Extra Work

Search Methods

Brute Force

Try all combinations for all regions.

Informed Search

Choose an element that does not conflict with neighbours.

General Search - Types

Instead of writing a custom program for every problem to be solved, general-purpose methods aim to write search algorithms into which individual problems can be plugged-in.

State Space Search / Solution Space Search

- Describe the given state
- Devise an operator to choose an action in each state
- Navigate the state space in search of the desired or goal state

State Space Search → Graph Search

- A state is a representation of a situation.
- A *MoveGen function* captures the moves that can be made in a given state.
- It returns a set of states – *neighbours* – resulting from the moves.
- The state space is an *implicit* graph defined by the MoveGen function.
- A *GoalTest function* checks if a given state is a *goal* state.

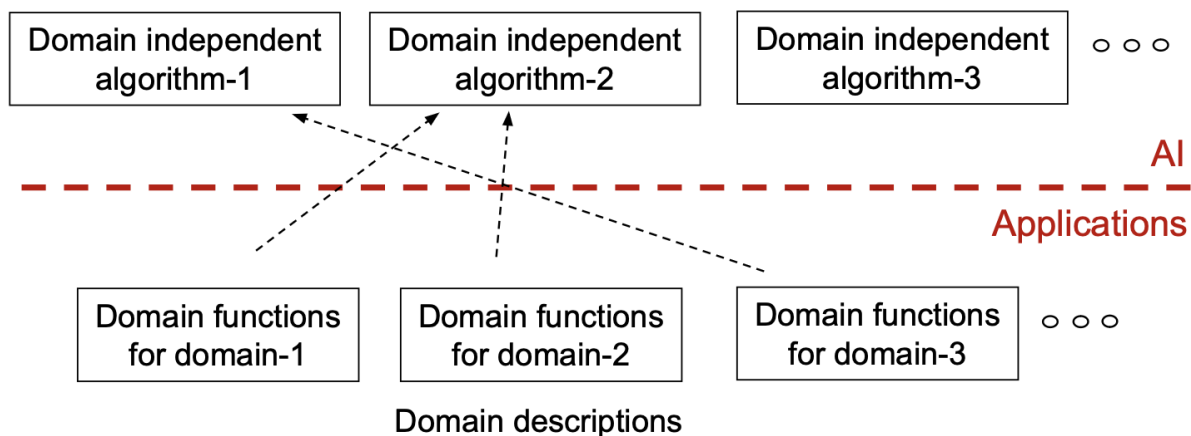
- A *search algorithm* navigates the state space using these two functions

Constraint Processing

- Define a set of variables each with their own domain
- Describe the constraints between subsets of variables
- Search for an assignment for each variable such that all constraints are satisfied

General Search → Algorithms

Develop general problem-solving algorithms in a domain-independent form.



The user needs to create a domain description and plug it into a suitable problem-solving algorithm for a domain.

Functions

- **MoveGen(N):**
 - A domain function that returns the neighbours of node N.
 - provided by the user.
 - A MoveGen function describes exactly one state space.
- **GoalTest(N):**
 - A domain function that tells you whether N is a goal node or not.

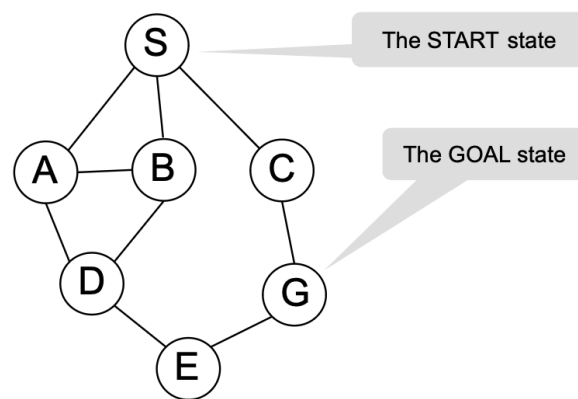
- provided by the user.
- **Generate and Test:**
 - Traverse the space by generating new nodes (using *MoveGen*)
 - test each node (using *GoalTest*) whether it is the goal or not

Example

The *MoveGen* function

```
S □ (A,B,C)
A □ (S,B,D)
B □ (S,A,D)
C □ (S,G)
D □ (A,B,E)
E □ (D,G)
G □ (C,E)
```

The State Space



We have represented the *set* of neighbours as a *list* returned by the *MoveGen* function

Simple Search Algorithm 1

SimpleSearch1()

```

1  OPEN □ {S}
2  while OPEN is not empty
3    do pick some node N from OPEN
4      OPEN □ OPEN - {N}
5      if GoalTest(N) = TRUE
6        then return N
7      else OPEN □ OPEN ∪ MoveGen(N)
8  return FAILURE
```

SimpleSearch1 simply picks a node N from OPEN and checks if it is the goal.

- If Yes it returns N
- If No it adds children of N to OPEN

Simple Search Algorithm 2

```
SimpleSearch2()
1  OPEN  $\leftarrow \{start\}$ 
2  CLOSED  $\leftarrow \{\}$ 
3  while OPEN is not empty
4      do Pick some node  $N$  from open
5          OPEN  $\leftarrow OPEN - \{N\}$ 
6          CLOSED  $\leftarrow CLOSED \cup \{N\}$ 
7          if GoalTest( $N$ ) = TRUE
8              then return  $N$ 
9          else OPEN  $\leftarrow OPEN \cup \{MoveGen(N) - CLOSED\}$ 
10 return FAILURE
```

When a node is picked from OPEN, add it to CLOSED

If a new node is on CLOSED do not add it to OPEN

SimpleSearch2 returns the goal node when it finds it.

What is *needed* in many problems is the *path* to the goal node.

Planning Problems



Goal is known or described, path is sought

Examples:

- Goal is known or described, path is sought
- River crossing problems
- Route finding problems

- Rubik's cube
- 8/15/24-puzzle
- Cooking a dish

Node Pairs

- The State Space is modeled as a graph with each node being a state.
- We need to keep track of the parents of each node *during search* so that we can reconstruct the path when we find the goal node.
- One way is to modify the search space node to store the entire path in the node itself.
- An elegant way is to store *node pairs* in the search space, where each node is a pair (currentNode, parentNode)

Deterministic Search Algorithms

- Use LIST data structures instead of SET
- Replace
“pick *some* node *N* from OPEN” with “pick node from head of OPEN”
- Instead of adding new nodes to OPEN as a SET insert them in a specified place in the list.
 - where you do so will determine the behaviour of the search algorithm

LIST Notations

1. $[]$ is an empty list.
2. $[]$ **is empty** = TRUE
3. $[1]$ **is empty** = FALSE
4. $LIST_2 \leftarrow ELEMENT : LIST_2$
5. $LIST_2 \leftarrow HEAD : TAIL$
6. $[1] = [1] : []$
7. $1 = \mathbf{head}[1] = \mathbf{head} \ 1 : []$ (Head is an element)
8. $[] = \mathbf{tail} \ [1] = \mathbf{tail} \ 1 : []$ (Tail is a list)

9. (**tail** [1]) **is empty** = TRUE
10. [3,2,1] = 3 : [2,1] = 3 : 2 : [1] = 3 : 2 : 1 : []
11. 3 = **head** [3,2,1]
12. [2,1] = **tail** [3,2,1]
13. 2 = **head tail** [3,2,1]
14. 1 = **head tail tail** [3,2,1]
15. LIST ₃ = LIST ₁ ++ LIST ₂
16. [] = [] ++ []
17. LIST = LIST ++ [] = [] ++ LIST
18. [o, u, t, r, u, n] = [o, u, t] ++ [r, u, n]
19. [r, u, n] ++ [o, u, t] = [r, u, n, o, u, t]
20. [r, o, u, t] = (**head** [r, u, n]) : [o, u, t]
21. [n, u, t] = **tail tail** [r, u, n] ++ **tail** [o, u, t]

TUPLE operators

1. (a, b) ← (101, 102)
2. pair ← (101, 102)
3. (a, b) ← pair
4. (a, _) ← pair
5. a ← **first** pair
6. (_, b) ← pair
7. b ← **second** pair
8. (_, _, c) ← (101, 'AI SMPS', 4)
9. c ← **third** (101, 'AI SMPS', 4)
10. 4 = **third** (101, 'AI SMPS', 4)

LIST and TUPLES

1. $101 = \text{head second}(1, [101, 102, 103], \text{null})$
2. $[102, 103] = \text{tail second}(1, [101, 102, 103], \text{null})$
3. $(a, h : t, c) = (1, [101, 102, 103], \text{null})$
 $a = 1$
 $h = 101$
 $t = [101, 102]$
 $c = \text{null}$

Depth First and Breadth First Search

Ancillary Functions

Remove Duplicates



Removes from nodeList any nodes that are already in OPEN or in CLOSED

REMOVESEEN(nodeList, OPEN, CLOSED)

- 1 **if** nodeList is empty
- 2 **return** empty list
- 3 **else** node $\leftarrow$ head nodeList
- 4 **if** OCCURSIN(node, OPEN) **or** OCCURSIN(node, CLOSED)
- 5 **return** REMOVESEEN(tail nodeList, OPEN, CLOSED)
- 6 **else return** node : REMOVESEEN(tail nodeList, OPEN, CLOSED)

Occurs in



Looks for node N as the first element of a nodePair in a list of nodePairs. Used for removing duplicates

OCCURSIIN(node, nodePairs)

```
1  if nodePairs is empty
2      return FALSE
3  elseif node = first head nodePairs
4      return TRUE
5  else return OCCURSIIN(node, tail nodePairs)
```

Make Pairs



Converts a list of nodes to a list of node pairs, with *parent* being the common parent of each node in list.

MAKEPAIRS(nodeList, parent)

```
1  if nodeList is empty
2      return empty list
3  else (head nodeList, parent) : MAKEPAIRS(tail nodeList, parent)
```

👉 *Makes a pair (head(nodeList), parent)*

For example:

$\text{makePairs}([A, B, C, D], S) = [(A,S), (B,S), (C,S), (D,S)]$

Reconstruct Path



Traces back through Parent links and reconstructs the path found.

```
RECONSTRUCTPATH(nodePair, CLOSED)
1  SKIPTO(parent, nodePairs)
2      if parent = first head nodePairs
3          return nodePairs
4      else return SKIPTO(parent, tail nodePairs)

5  (node, parent) ← nodePair
6  path ← node : [ ]
7  while parent is not null
8      path ← parent : path
9      CLOSED ← SKIPTO(parent, CLOSED)
10     ( —, parent) ← head CLOSED
11 return path
```

Depth First Search

Breadth First Search



OPEN = STACK



OPEN = QUEUE

DFS(S)

```

1 OPEN ← (S, null) : []
2 CLOSED ← empty list
3 while OPEN is not empty
4   nodePair ← head OPEN
5   (N, _) ← nodePair
6   if GOALTEST(N) = TRUE
7     return RECONSTRUCTPATH(nodePair, CLOSED)
8   else CLOSED ← nodePair : CLOSED
9   children ← MOVEGEN(N)
10  newNodes ← REMOVESEEN(children, OPEN, CLOSED)
11  newPairs ← MAKEPAIRS(newNodes, N)
12  OPEN ← newPairs ++ (tail OPEN)
13 return empty list

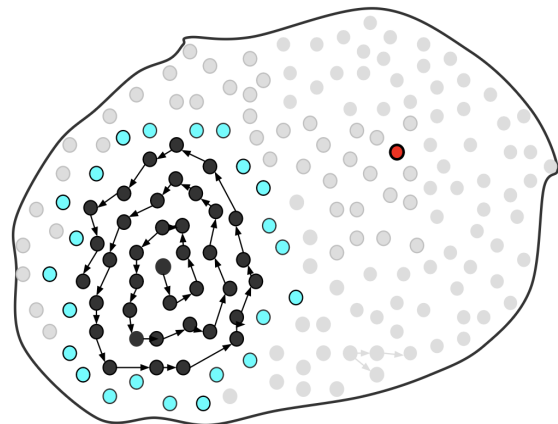
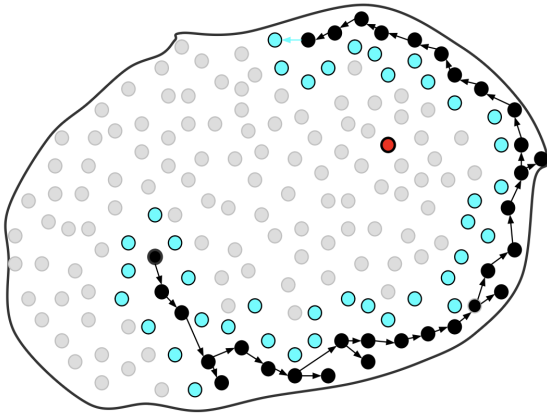
```

BFS(S)

```

1 OPEN ← (S, null) : []
2 CLOSED ← empty list
3 while OPEN is not empty
4   nodePair ← head OPEN
5   (N, _) ← nodePair
6   if GOALTEST(N) = TRUE
7     return RECONSTRUCTPATH(nodePair, CLOSED)
8   else CLOSED ← nodePair : CLOSED
9   children ← MOVEGEN(N)
10  newNodes ← REMOVESEEN(children, OPEN, CLOSED)
11  newPairs ← MAKEPAIRS(newNodes, N)
12  OPEN ← (tail OPEN) ++ newPairs
13 return empty list

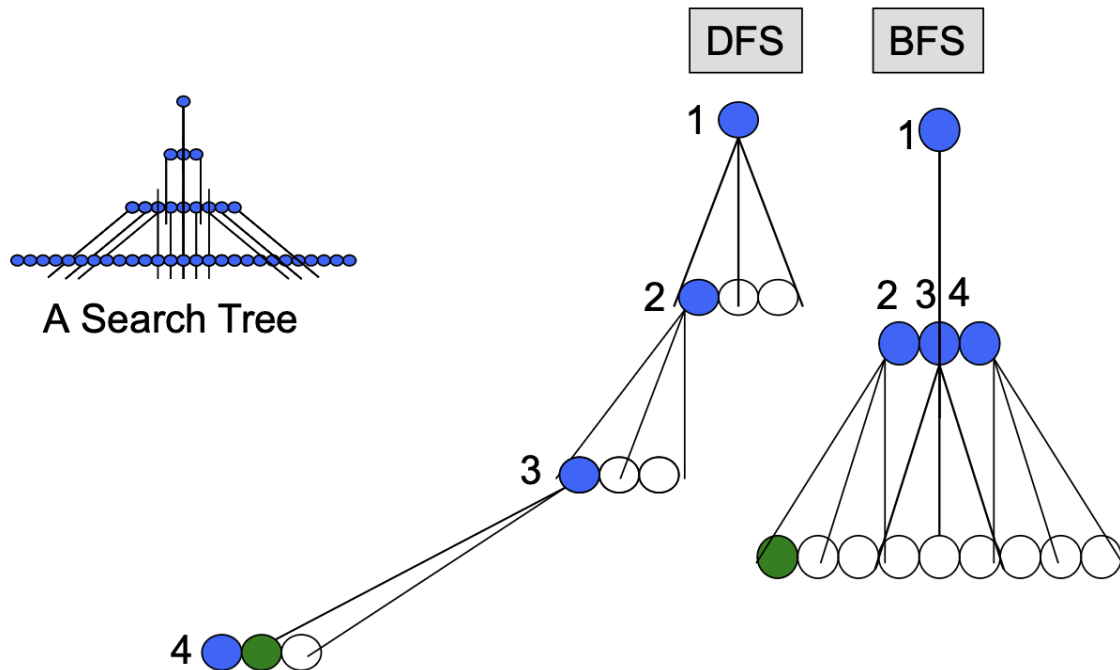
```



Examples with steps

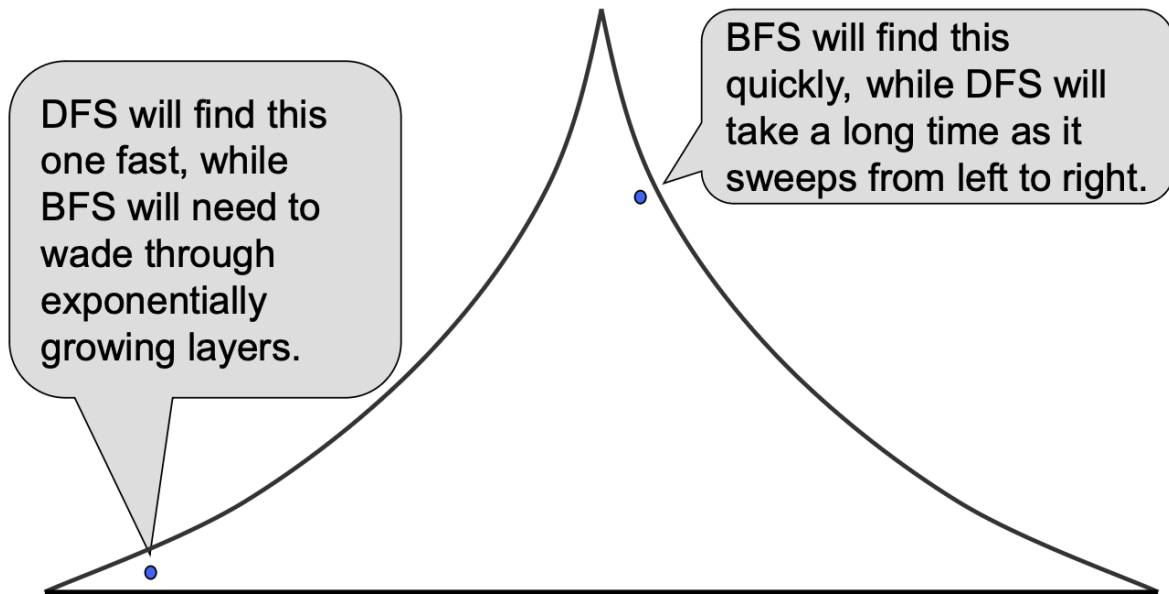
Analysis of BFS and DFS

Behaviour



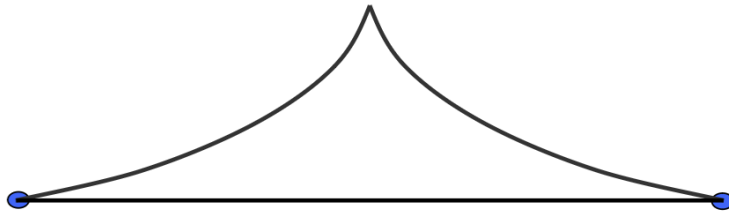
When the two searches pick the fifth node, coloured green, they have explored different parts of the search tree, expanding the four nodes before in the order shown.

Running Time



Time Complexity

- Assume constant branching fact b
- Assume that goal occurs somewhere at depth d
- Let N_{DFS} be the number of nodes inspected by DFS
- Let N_{BFS} be the number of nodes inspected by BFS



$$N_{DFS} = d+1$$

$$N_{BFS} = (b^d - 1)/(b - 1) + 1$$

$$N_{DFS} = N_{BFS} = (b^{d+1} - 1)/(b - 1)$$

On the average

$$N_{DFS} = \frac{(d+1) + (b^{d+1} - 1)/(b - 1)}{2} \approx \frac{b^d}{2}$$

$$N_{BFS} = \frac{(b^d - 1)/(b - 1) + 1 + (b^{d+1} - 1)/(b - 1)}{2} \approx \frac{b^d(b+1)}{2(b-1)}$$

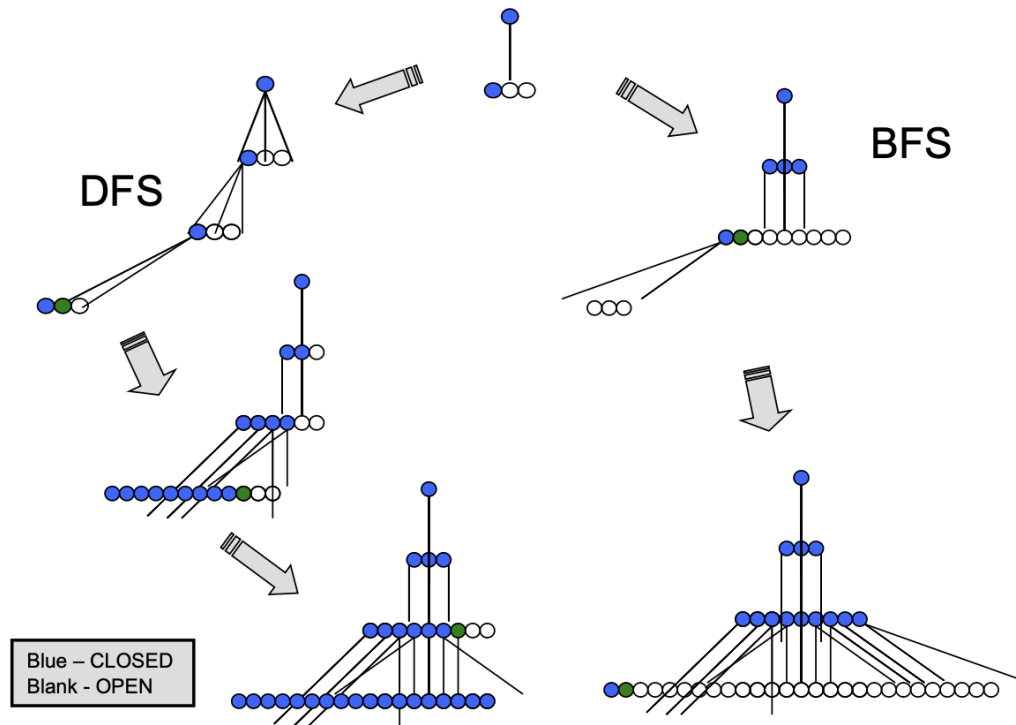
$$N_{BFS} \approx N_{DFS} \frac{(b+1)}{(b-1)}$$



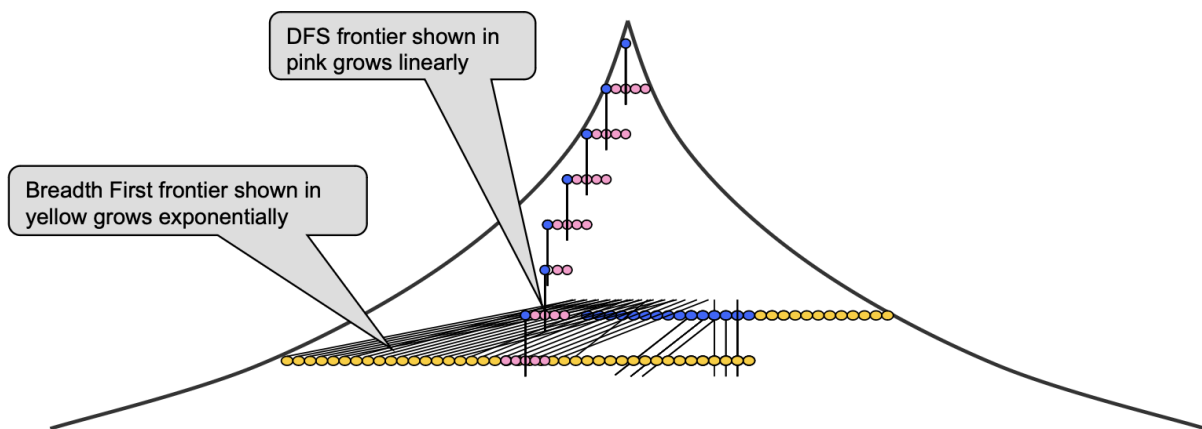
Average time complexity is exponential

Space

(Size of OPEN)



Search Frontiers



The number of nodes on the search frontier is an indication of space requirement.

Comparison - Summary

	Depth First Search	Breadth First Search
Time	Exponential	Exponential
Space	Linear	Exponential
Quality of solution	No guarantees	Shortest path
Completeness	Not for infinite search space	Guaranteed to terminate if solution path exists

Best of Both

Depth Bounded DFS (DBDFS)

Do DFS with a depth bound d .

- Linear space
- Not complete
- Does not guarantee shortest path

Algorithm

DB-DFS(S, depthBound)

```
1  OPEN ← (S, null, 0) : [ ]
2  CLOSED ← empty list
3  while OPEN is not empty
4      nodePair ← head OPEN
5      (N, —, depth) ← nodePair
6      if GOALTEST(N) = TRUE
7          return RECONSTRUCTPATH(nodePair, CLOSED)
8      else CLOSED ← nodePair : CLOSED
9          if depth < depthBound
10             children ← MOVEGEN(N)
11             newNodes ← REMOVESEEN(children, OPEN, CLOSED)
12             newPairs ← MAKEPAIRS(newNodes, N, depth + 1)
13             OPEN ← newPairs ++ tail OPEN
14         else OPEN ← tail OPEN
15  return empty list
```

Store depth information in search node

DBDFS -2

DB-DFS-2(S, depthBound)

```
1  count ← 0
2  OPEN ← (S, null, 0) : [ ]
3  CLOSED ← empty list
4  while OPEN is not empty
5      nodePair ← head OPEN
6      (N, —, depth) ← nodePair
7      if GOALTEST(N) = TRUE
8          return (count, RECONSTRUCTPATH(nodePair, CLOSED))
9      else CLOSED ← nodePair : CLOSED
10     if depth < depthBound
11         children ← MOVEGEN(N)
12         newNodes ← REMOVESEEN(children, OPEN, CLOSED)
13         newPairs ← MAKEPAIRS(newNodes, N, depth + 1)
14         OPEN ← newPairs ++ tail OPEN
15         count ← count + length newPairs
16     else OPEN ← tail OPEN
17  return (count, empty list)
```

returns count of nodes visited

Depth First Iterative Deepening (DFID)

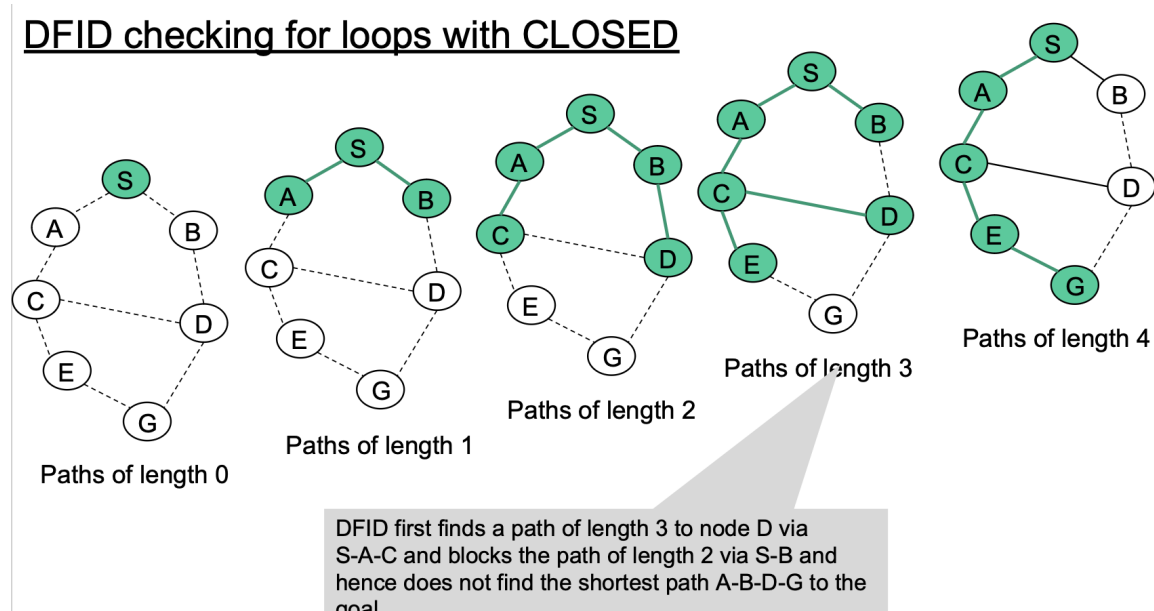
DFID(S)

```

1  count  $\leftarrow$  -1
2  path  $\leftarrow$  empty list
3  depthBound  $\leftarrow$  0
4  repeat
5      previousCount  $\leftarrow$  count
6      (count, path)  $\leftarrow$  DB-DFS-2(S, depthBound)
7      depthBound  $\leftarrow$  depthBound + 1
8  until (path is not empty) or (previousCount = count)
9  return path

```

Closed nodes



Without closed nodes



Without CLOSED DFID finds the shortest path

- We still need the parent nodes to reconstruct the path
- We can still store *nodePairs* in CLOSED, and use them for reconstruction but have to be careful to choose the right *nodePair* as nodes can be reached through multiple parents
- Perhaps we can store the entire path at each node?
- For configuration problems the path is not needed. In fact the shortest “path” may have no importance except time complexity.
- But DFID enables *anytime* move generation in games like chess
 - DFID was invented by chess programmers

DepthFirstIterativeDeepening(*start*)

```

1  depthBound  $\square$  1
2  while TRUE
3      do DepthBoundedDFS(start, depthBound)
4      depthBound  $\square$  depthBound + 1

```

DFID does a series of *DBDFS*s with increasing depth bounds

▼ Cost of Extra Work

DFID: cost of extra work

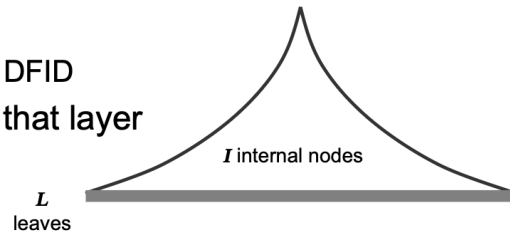
For every new layer in the search tree that DFID explores, it searches the **entire tree up to that layer all over again**.

DFID inspects $(L+I)$ nodes whereas
Breadth First Search would have inspected L nodes

$$N_{\text{DFID}} = N_{\text{BFS}} \frac{(L+I)}{L}$$

But $L = (b-1) \cdot I + 1$ for a full tree with branching factor b

$$\therefore N_{\text{DFID}} \approx N_{\text{BFS}} \frac{b}{(b-1)} \quad \text{for large } b$$



The extra cost is not significant!

Week 3

Heuristic Search

Idea

Heuristic Function

Best-First Search

Algorithm

Search Frontiers

Hill Climbing

Algorithm

Space

Effect of Density on Hill Climbing

Travelling Salesman Problem (TSP)

Greedy Heuristic

Nearest neighbour Heuristic

Escaping local optimum

Solution Space Search

Synthesis vs Perturbation

Exploration and Exploitation

Beam Search

Variable Neighbourhood Descent

Best Neighbour - variation of Hill Climbing

Tabu Search

- Both Depth-first search (DFS) and Breadth-first searches (BFS) are oblivious to the goal.
- Irrespective of where the goal is in the state space both algorithms set out on the same predetermined trajectories every time.

Heuristic Search

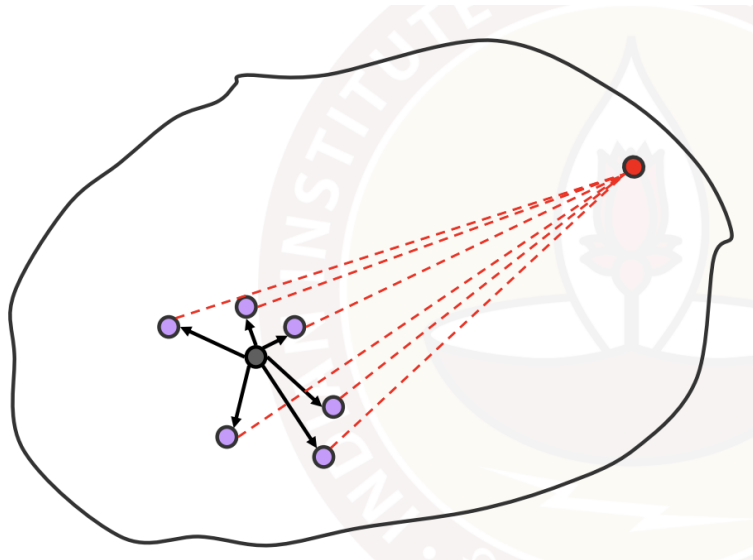
Idea

- Testing the neighbourhood and following the steepest gradient identifies which of the neighbours is the lowest.
- The idea of a heuristic function is that it takes a state or a node as input and computes a number which is an estimate of the distance of that node from the goal.

- Then a search algorithm can pick that node from OPEN which has the lowest heuristic value.
- The heuristic function $h(N)$ is typically a user defined function.
 - in addition to the MoveGen and the GoalTest functions
- Since $h(\text{goal}) = 0$ we are effectively seeking to minimize $h(N)$.

Heuristic Function

- informed search
- goal-directed
- The heuristic function estimates the distance to the goal.
- This estimate, $h(n)$, can be used to decide which node to pick from OPEN.

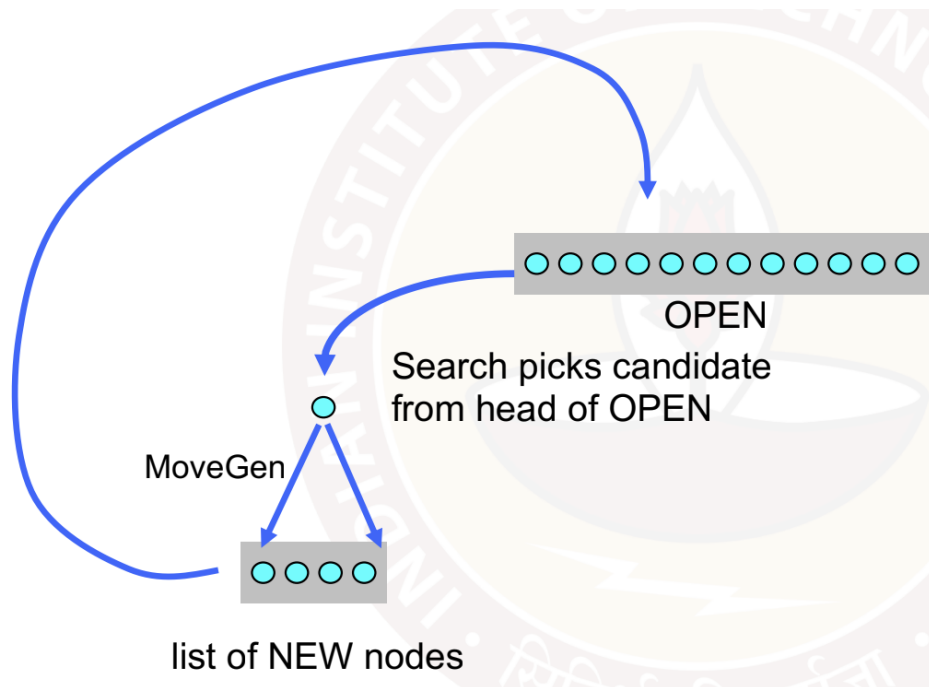


Best-First Search



Best First Search inserts new candidates into OPEN sorted on $h(n)$.

OPEN = PRIORITY QUEUE



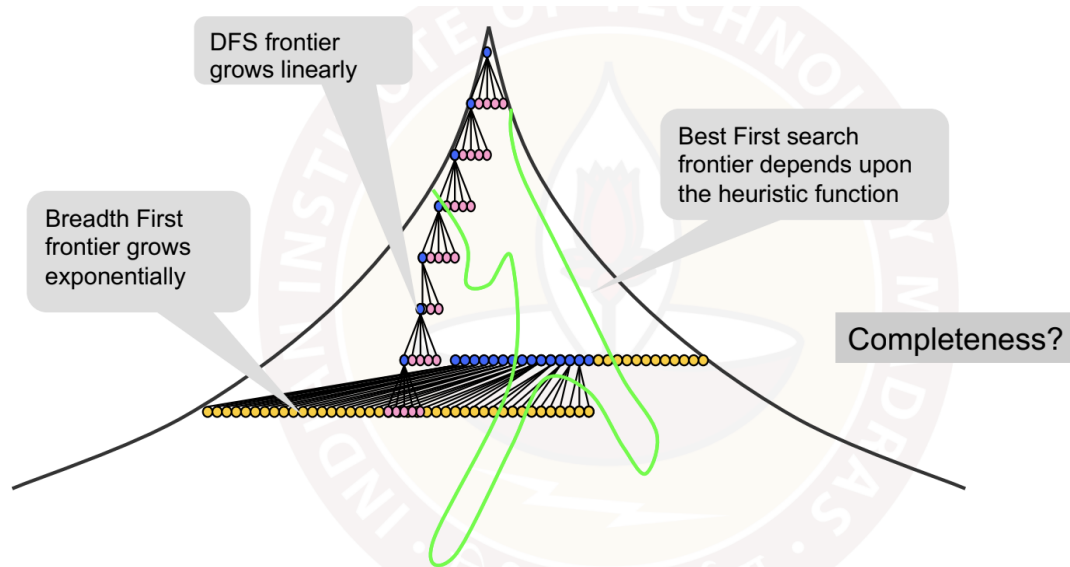
Algorithm

BEST-FIRST-SEARCH(S)

- 1 $OPEN \leftarrow (S, \text{null}, h(S)) : []$
- 2 $CLOSED \leftarrow \text{empty list}$
- 3 **while** $OPEN$ is not empty
- 4 $\text{nodePair} \leftarrow \text{head } OPEN$
- 5 $(N, _, _) \leftarrow \text{nodePair}$
- 6 **if** $\text{GOALTEST}(N) = \text{TRUE}$
- 7 **return** $\text{RECONSTRUCTPATH}(\text{nodePair}, CLOSED)$
- 8 **else** $CLOSED \leftarrow \text{nodePair} : CLOSED$
- 9 $\text{children} \leftarrow \text{MOVEGEN}(N)$
- 10 $\text{newNodes} \leftarrow \text{REMOVESEEN}(\text{children}, OPEN, CLOSED)$
- 11 $\text{newPairs} \leftarrow \text{MAKEPAIRS}(\text{newNodes}, N)$
- 12 $OPEN \leftarrow \text{sort}_h(\text{newPairs} ++ \text{tail } OPEN)$
- 13 **return empty list**

Search Frontiers

The Search Frontier is an indication of space requirement.



Hill Climbing



a local search algorithm and a constant space algorithm

Algorithm

Move to the best neighbour if it is better, else terminate

```
Hill Climbing
node ← Start
newNode ← head(sorth moveGen (node)))
While h(newNode) < h(node) Do
    node ← newNode
    newNode ← head(sorth (moveGen (node))))
endWhile
return node
End
```

Algorithm Hill Climbing

In practice sorting is not needed, only the best node

Change of termination criterion

Local search – Hill Climbing has burnt its bridges by not storing OPEN

Space

- HC only looks at local neighbours of a node. It's space requirement is thus **constant!**
 - A vast improvement on the exponential space for BFS and DFS.
- HC only moves to a better node. It terminates if it cannot. Consequently the **time complexity is linear**.
- Its **termination criterion is different**. It stops when no better neighbour is available. It treats the problem as an **optimization problem**.
- However, it is not complete, and **may not find the global optimum** which corresponds to the solution!

Effect of Density on Hill Climbing



Hill Climbing

- Inspect all neighbours
- Move to the best neighbour if it is better than current node

Hill climbing gets stuck on a local optimum when none of the neighbours is better than the current node.

- The denser the neighbourhood the less likely this is, but
- the denser the neighbourhood the more the number of neighbours that has to be inspected at each step

Travelling Salesman Problem (TSP)

Given a set of cities and given a distance measure between every pair of cities, the task is to find a Hamiltonian cycle, visiting each city exactly once, having least cost.

In a tour of N cities,

- every node has exactly 2 edges connected to it
- there are exactly N edges in the tour
- NP-Hard

Greedy Heuristic

- Sort the edges
- Add shortest available edge to the tour
 - as long as it does not close the loop prematurely

Nearest neighbour Heuristic

- Start at some city.
- Move to nearest neighbour
 - as long as it does not close the loop prematurely

Escaping local optimum

Given that

- it is difficult to define heuristic functions that are monotonic and well behaved

the alternative is

- to look for algorithms that can do better than Hill Climbing.

We look at three deterministic methods next,
and will look at randomized methods later

First we look at searching in

- the solution space or plan space

Solution Space Search

- By solution space search we mean a formulation of the search problem such that when we find the goal node we have the solution and we are done.
- **We do not have to reconstruct the path.**
- Configuration problems fall easily into this category.
- Every node in the search space is a *candidate solution*.
- A candidate is a solution if it satisfies the goal description.

Note:

- Planning problems can be solved with solution space search too.
- Then we say we are searching in the plan space.

Synthesis vs Perturbation

- **Synthesis methods** are constructive methods. Starting with the initial state we build the solution piece by piece.
- **Perturbation:** Permuting the list results in new candidates

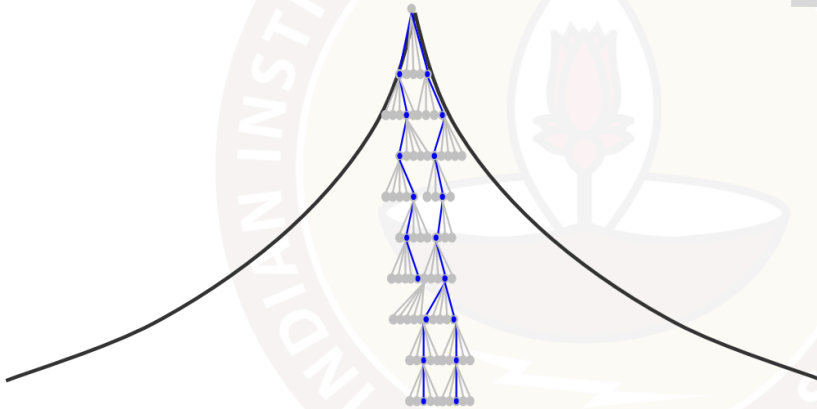
Exploration and Exploitation

- **Exploitation** pulls the search along the gradient
- **Exploration** leads it away from the trodden path

Beam Search

Look at more than one option at each level.
For a beam width b , look at best b options.

After all computer
memory is becoming
cheaper



Beam Search with beam width = 2

Variable Neighbourhood Descent

VariableNeighbourhoodDescent()

1 $node \leftarrow start$

2 **for** $i \leftarrow 1$ **to** n

3 **do** $moveGen \leftarrow MoveGen(i)$

4 $node \leftarrow HillClimbing(node, moveGen)$

5 **return** $node$

The algorithm assumes that the function $moveGen$ can be passed as a parameter. It assumes that there are N $moveGen$ functions sorted according to the density of the neighbourhoods produced.

Best Neighbour - variation of Hill Climbing

Best Neighbour
N $\leftarrow$ Start
bestSeen $\leftarrow$ N
Until *some termination criterion*
 N $\leftarrow$ best(moveGen (N))
 IF N *better than* bestSeen
 bestSeen $\leftarrow$ N
return bestSeen
End

Move to the best neighbour *anyway*

Observe: Algorithm needs external criterion to terminate

Tabu Search

Best Neighbour
N $\leftarrow$ Start
bestSeen $\leftarrow$ N
Until *some termination criterion*
 N $\leftarrow$ best(allowed(moveGen (N)))
 IF N *better than* bestSeen
 bestSeen $\leftarrow$ N
return bestSeen
End

Move to the best neighbour anyway,
but only if it is not tabu

The word *tabu* comes from the Tongan word to indicate things that cannot be touched because they are sacred. - *Wikipedia*

If a tabu move results in a node that is better than bestSeen then make an exception for it (that is, add it to allowed set).

Week 4

The SAT Problem

Conjunctive Normal Form (CNF)

Complexity of SAT

2SAT

3SAT

Probability of 3SAT Variables being satisfied

Iterated Hill Climbing

Algorithm

Reaching the Global Maximum

Stochastic Local Search

Random Walk - Pure Exploration

Stochastic Hill Climbing

Sigmoid Function

Annealing

Simulated Annealing

Effect of T on Probability

Why Simulated Annealing works

Simulated Annealing works well when

Population Based Methods

Genotypes and Phenotypes

Genetic Algorithms

Artificial Selection

Crossover Operators

Algorithm

Solving Travelling Salesman Problem (TSP) with Genetic Algorithms (GA)

Crossovers

Crossover Operators

Adjacency Representation

Ordinal Representation

Ant Colony Optimisation

Algorithm

Updating Pheromone

The SAT Problem

SAT: Satisfied



Given a set of Boolean or Propositional variables each taking a value from {true, false} or {1, 0}, and a formula F involving these variables, to find a valuation for the variables such that the formula F is satisfied (becomes true)

Conjunctive Normal Form (CNF)

Each clause has to be satisfied or made true.

For example,

$$F = (a \vee \neg b \vee e) \wedge b \wedge (\neg c \vee d \vee f) \wedge (d \vee \neg e) \wedge (\neg a \vee b \vee d) \wedge (a \vee \neg d)$$

For N variables there are 2^N candidates (the state space)

Complexity of SAT

2SAT

- problems where each clause has at most two literals
- known to be in P

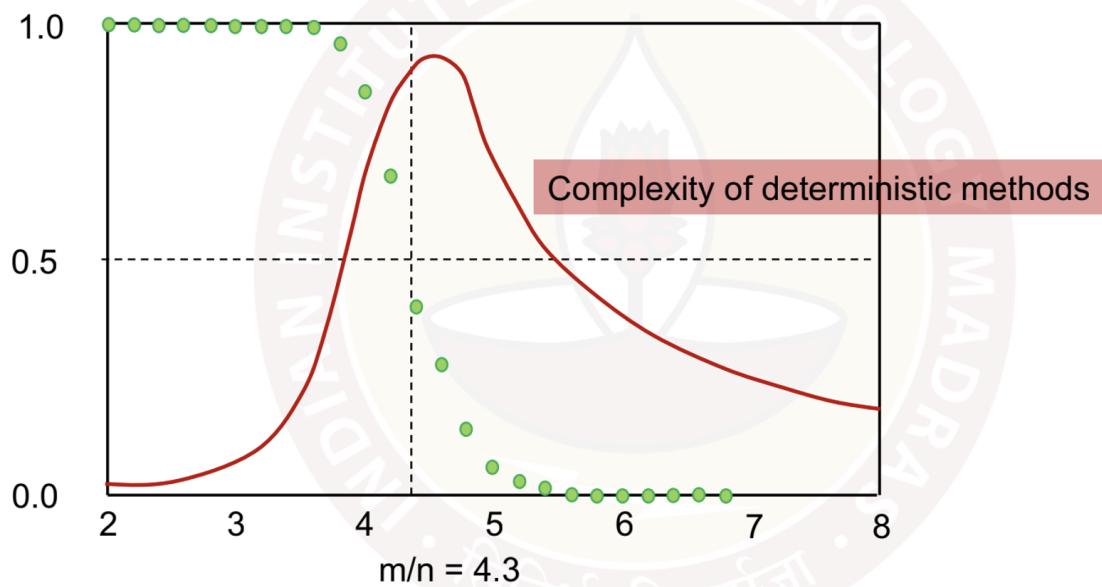
3SAT

- problems where each clause has at most three literals
- known to be NP-complete

Probability of 3SAT Variables being satisfied

m: clauses

n: variables



Iterated Hill Climbing

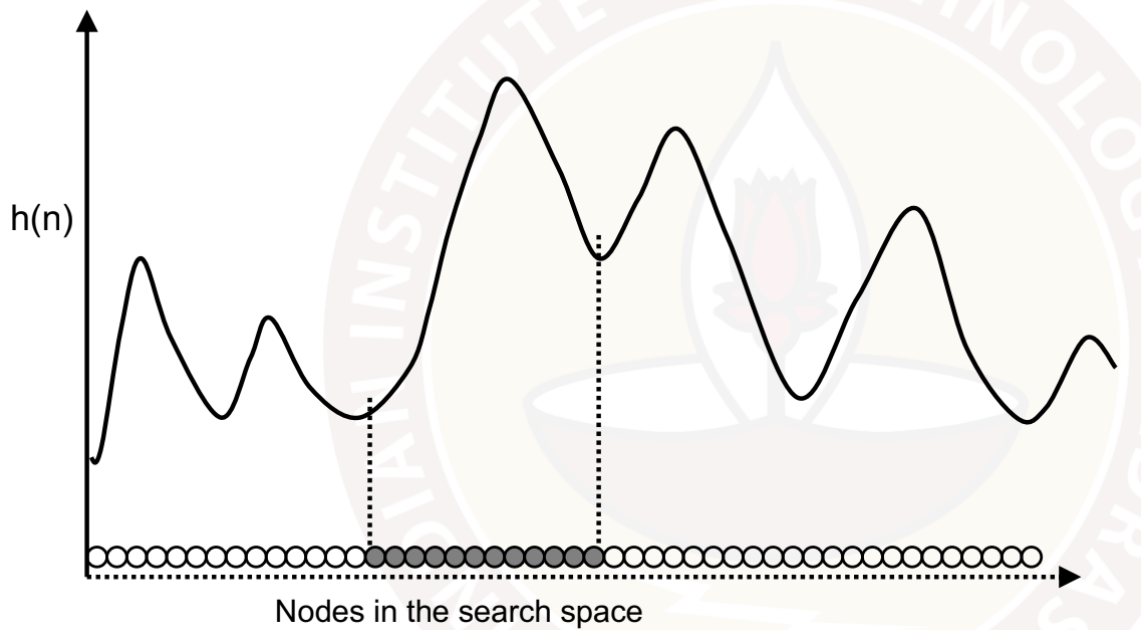
Iterated Hill Climbing reaches solution from any of the nodes as the starting point

Algorithm

ITERATED-HILL-CLIMBING(N)

- 1 $\text{bestNode} \leftarrow \text{random candidate solution}$
- 2 **repeat** N **times**
- 3 $\text{currentBest} \leftarrow \text{HILL-CLIMBING}(\text{new random candidate solution})$
- 4 **if** $h(\text{currentBest})$ **is better than** $h(\text{bestNode})$
- 5 $\text{bestNode} \leftarrow \text{currentBest}$
- 6 **return** bestNode

Reaching the Global Maximum



A random starting point from any of the shaded nodes would lead to the global maximum.

Stochastic Local Search

Random Walk - Pure Exploration



RandomWalk explores the search space in a random fashion.

RandomWalk()

```

1  node ← random candidate solution or start
2  bestNode ← node
3  for i ← 1 to n
4      do node ← RandomChoose(MoveGen(node))
5          if node is better than bestNode
6              then bestNode ← node
7  return bestNode

```

Stochastic Hill Climbing

Let a stochastic local search algorithm be at the current node v_C .

Select a random neighbour v_N

$$\Delta E = (\text{eval}(v_N) - \text{eval}(v_c))$$

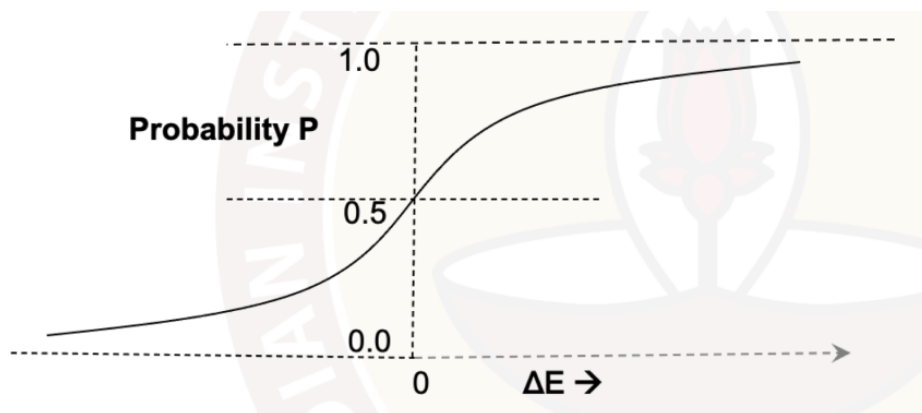
- If ΔE is **positive**, it means that v_N is **better than** v_C
 - The algorithm should move to it with a **high probability**.
- If ΔE is **negative**, it means that v_N is **worse than** v_C
 - The algorithm should move to it with a **low probability**.

Sigmoid Function



combines exploration with exploitation

- Accept a good random move with high probability
- Accept a bad random move too but with low probability



$$\text{Probability } P = \frac{1}{1 + e^{\frac{-\Delta E}{T}}}$$

where, $\Delta E = (\text{eval}(v_N) - \text{eval}(v_c))$

for a maximization problem and T is a parameter.

Annealing

In metallurgy and materials science, annealing is a heat treatment that alters the physical and sometimes chemical properties of a material to increase its ductility and reduce its hardness, making it more workable.

It involves heating a material above its recrystallization temperature, maintaining a suitable temperature for an appropriate amount of time and then cooling.

In annealing, atoms migrate in the crystal lattice and the number of dislocations decreases, leading to a change in ductility and hardness. As the material cools it recrystallizes.

Simulated Annealing

```
SimulatedAnnealing()
1  node ← random candidate solution or start
2  bestNode ← node
3  T ← some large value
4  for time ← 1 to numberOfEpochs
5      do while some termination criteria /* M cycles in a simple case */
6          do
7              neighbour ← RandomNeighbour(node)
8               $\Delta E \leftarrow Eval(neighbour) - Eval(node)$ 
9              if Random(0, 1) < 1 / (1 + e- $\Delta E/T$ )
10                 then node ← neighbour
11                 if Eval(node) > Eval(bestNode)
12                     then bestNode ← node
13     T ← CoolingFunction(T, time)
14 return bestNode
```

- Function *CoolingFunction* lowers the temperature after each.
- Function *Random(0,1)* generates a random number in the range 0 to 1 with uniform probability.

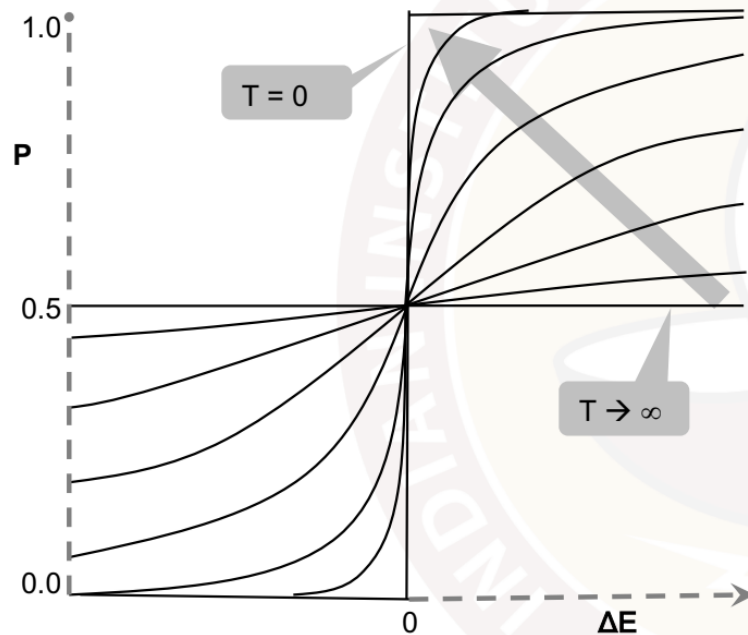
Effect of T on Probability

- The probability curves (Sigmoid function) for different values of T.
- The probability of making a move increases as ΔE increases.
- For very large T the algorithm behaves like Random Walk. $P = 0.5$.

- For $T = 0$ the behaviour becomes deterministic like the Hill Climbing algorithm.

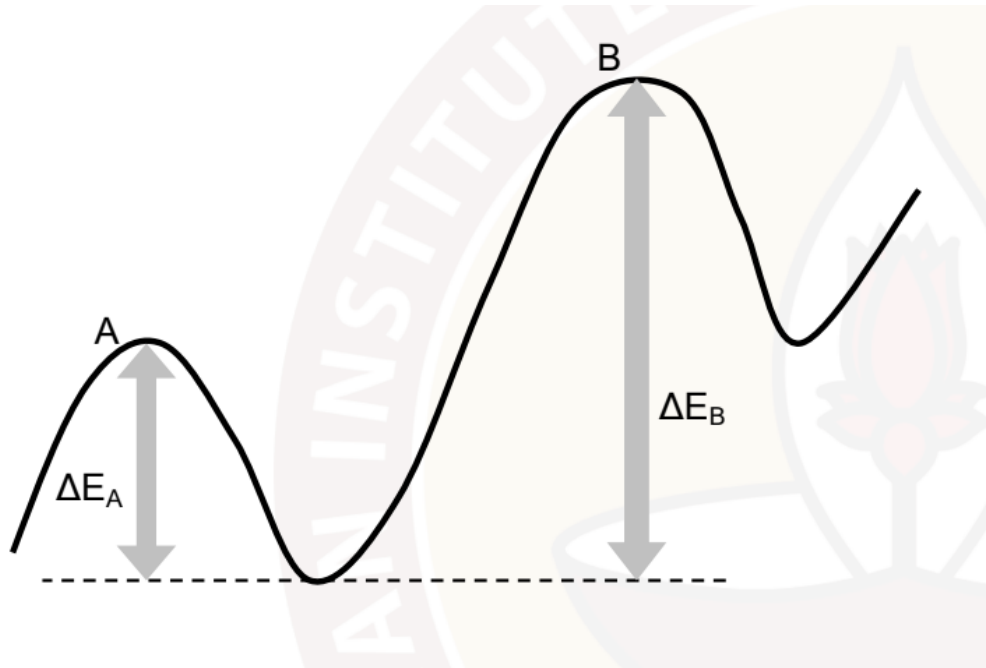


As T decreases the moves become more exploitative.



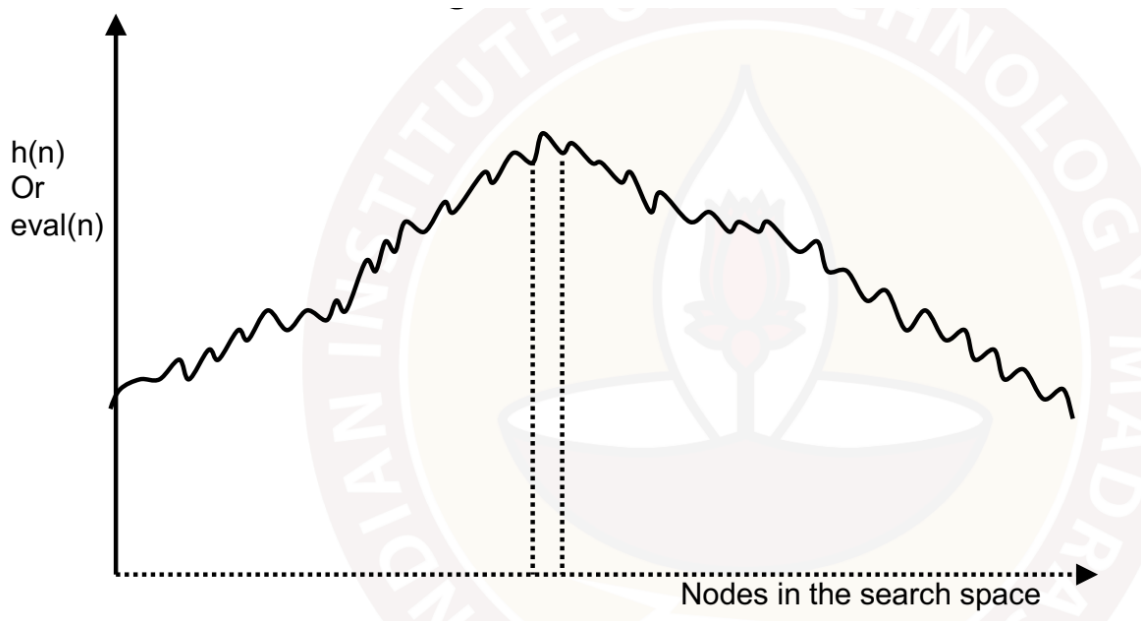
Why Simulated Annealing works

To move from A to B the algorithm has to incur a smaller loss than to move from B to A. Simulated Annealing is more likely to move from A to B than vice versa.



Simulated Annealing works well when

A jagged surface with many local optima but a broader trend towards the optimum is well suited for Simulated Annealing.



Observe that the footprint of Hill Climbing is very small.

Population Based Methods

Genotypes and Phenotypes

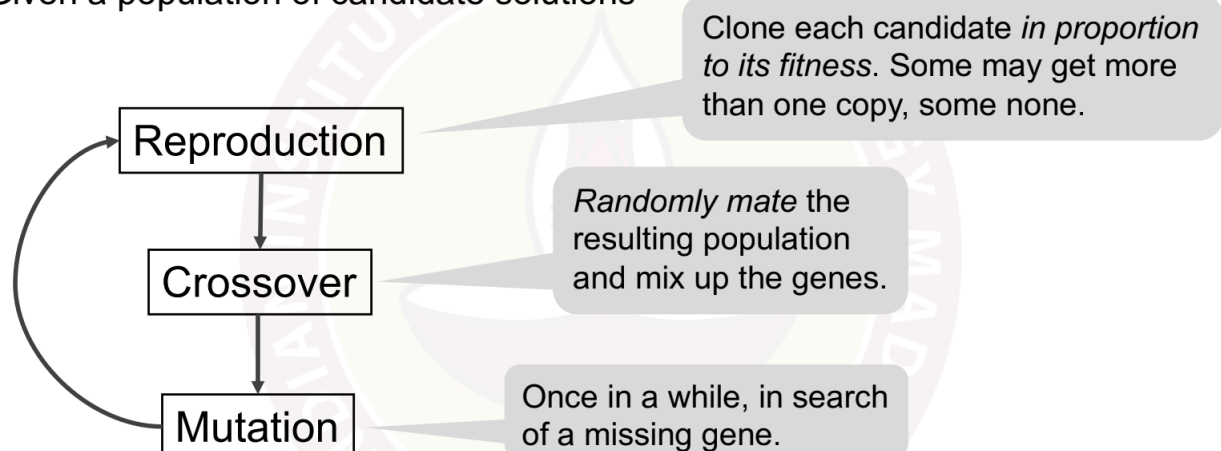
- An organism's genotype is the set of genes in its DNA responsible for a particular trait.
 - Mixing up genes via sexual reproduction
- An organism's phenotype is the physical expression of those genes.
 - Survival of the fittest via natural selection

Genetic Algorithms

- Devised by John Holland (1975), popularized by his student David Goldberg via his book “Genetic Algorithms”.
- A class of methods for optimization problems, more generally known as Evolutionary Algorithms.
- Heuristic stochastic adaptive search algorithms
- Implemented on a population of candidates in the solution space
- Inspired by the process of natural selection
- A fitness function evaluates each candidate
- The fittest candidates get to mate and reproduce

Artificial Selection

Given a population of candidate solutions



Crossover Operators

A single point crossover simply cuts the two parents at a randomly chosen point and recombines them to form two new solution strings.

$P_1 = X_1 X_2 X_3 X_4 X_5 X_6 X_7 X_8$
 $P_2 = Y_1 Y_2 Y_3 Y_4 Y_5 Y_6 Y_7 Y_8$

$C_1 = X_1 X_2 X_3 X_4 Y_5 Y_6 Y_7 Y_8$
 $C_2 = Y_1 Y_2 Y_3 Y_4 X_5 X_6 X_7 X_8$

For example, SAT

Any operator that mixes up the genes will do.
One can have multi-point crossovers, for example,

$P_1 = X_1 X_2 X_3 X_4 X_5 X_6 X_7 X_8$
 $P_2 = Y_1 Y_2 Y_3 Y_4 Y_5 Y_6 Y_7 Y_8$



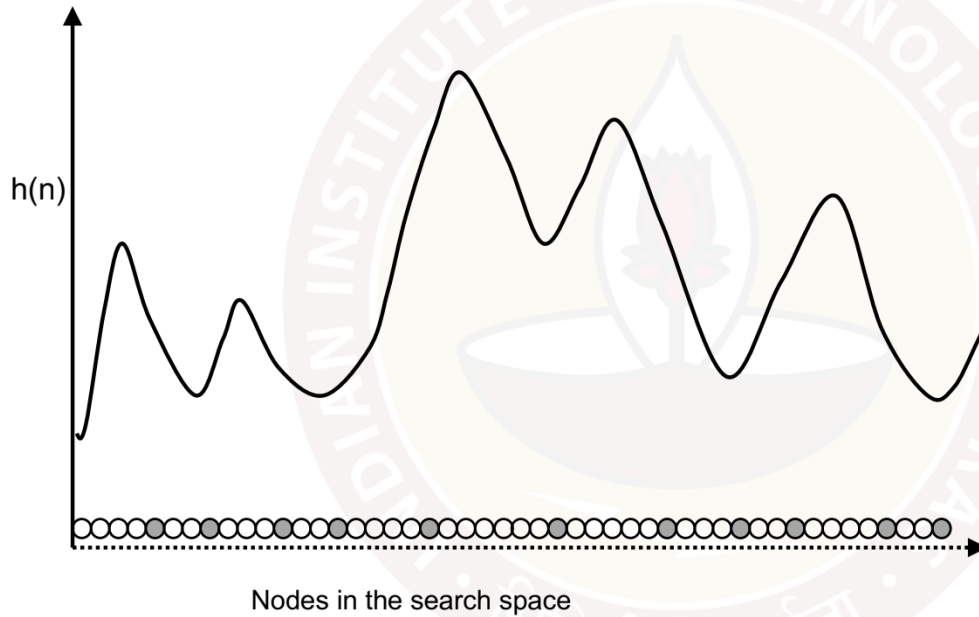
$C_3 = X_1 Y_2 X_3 Y_4 X_5 Y_6 X_7 Y_8$
 $C_4 = Y_1 X_2 Y_3 X_4 Y_5 X_6 Y_7 X_8$

Algorithm

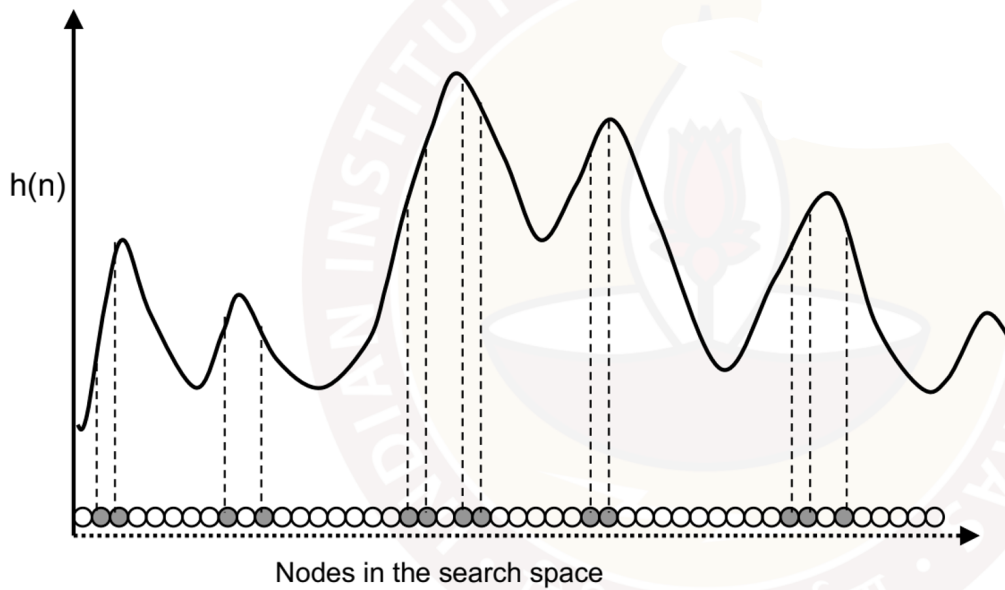
GENETIC-ALGORITHM()

- 1 $P \leftarrow$ create N candidate solutions ▷ initial population
- 2 repeat
- 3 compute fitness value for each member of P
- 4 $S \leftarrow$ with probability proportional to fitness value,
 randomly select N members from P
- 5 offspring $\leftarrow$ partition S into two halves, and randomly mate
 and crossover members to generate N offsprings
- 6 with a low probability mutate some offsprings
- 7 replace k weakest members of P with k strongest offsprings
- 8 until some termination criteria
- 9 return the best member of P

Initial Population



Evolved Population



The initial population may be randomly distributed, but as Genetic Algorithm is run the population has more members around the peak.

Solving Travelling Salesman Problem (TSP) with Genetic Algorithms (GA)

Create a population of candidate TSP solutions. Let the fitness function be the cost of the tour. It is a minimization problem.

In the Path Representation the tour is represented by a permutation of the cities, with the assumption that one returns from the last city in the permutation to the first.

- **Selection:**
 - Clone each tour in proportion to fitness.
 - The cheapest tours are the fittest.
- **Crossover:**
 - Randomly pair the resulting population and perform crossover.
- **Mutation:**
 - Randomly permute a tour once in a while

Crossovers

- Single point crossover does not work.

Crossover Operators

- Cycle crossover
- Partially Mapped Crossover (PMX)
- Order Crossover

Adjacency Representation

- Alternating Edges Crossover
 - Construct a child as follows
 - From a given city X choose the next city Y from P1
 - From the city Y choose the next city from P2
 - and so on...

- Heuristic Crossover
 - For each city choose the next from that parent (P1 or P2) whichever is closer

Ordinal Representation

The advantage of using the Ordinal Representation is that the Single Point Crossover produces valid offspring.

Ant Colony Optimisation

- Ant Colony Optimization is an optimization method that takes inspiration by the bio-semiotic (the study of signs and symbols and their use or interpretation) communication between ants
- Each ant constructs a solution using a stochastic greedy method using a combination of a heuristic function and pheromone trail following
- Related to the class of algorithms known as Swarm Optimization
- Applicable to problems that can be reduced to graph search
- Presented by Marco Dorigo in 1992

Algorithm

TSP-ACO()

```

1  bestTour ← nil
2  repeat
3      randomly place M ants on N cities
4      for each ant a      ▷ construct tour
5          for n ← 1 to N
6              ant a selects an edge from the distribution  $P_n^a$ 
7          update bestTour
8      for each ant a      ▷ update pheromone
9          for each edge (u, v) in the ant's tour
10             deposit pheromone  $\propto 1/\text{tour-length}$  on edge (u, v)
11 until some termination criteria
12 return bestTour

```

From a city i the k^{th} ant moves to city j with a probability given by,

$$P_{ij}^k(t) = \begin{cases} \frac{[\tau_{ij}(t)]^\alpha * [\eta_{ij}]^\beta}{\sum_{h \in \text{allowed}_k(t)} ([\tau_{ih}(t)]^\alpha * [\eta_{ih}]^\beta)} & \text{if } j \in \text{allowed}_k(t) \text{ the cities ant}_k \text{ is allowed to move to} \\ 0 & \text{otherwise} \end{cases}$$

Where, $\tau_{ij}(t)$ is pheromone on edge _{ij}
and η_{ij} is called visibility which
is inversely proportional
to the distance between cities i and j .

Updating Pheromone

- After constructing a tour in n time steps each ant k deposits an amount of pheromone $\frac{Q}{L_k}$ on the edges it has traversed

- which is inversely proportional to the cost of the tour L_k it found. Q is an appropriate constant.
- Total pheromone deposited on edge_{ij} is $\Delta\tau_{ij}(t, t + \eta)$
- The total pheromone on edge_{ij} is updated as

$$\tau_{ij}(t + \eta) = (1 - \rho) * \tau_{ij}(t) + \Delta\tau_{ij}(t, t + \eta)$$

where ρ is the rate of evaporation of pheromone